



EE2030

SEMESTER 3 - Group Project

Group Members

E/23/135

E/23/138

E/23/145

E/23/150

E/23/151

E/23/158

Week 01

Week 1 – Initial Study and Logic Design

In Week 1, we first analyzed the basic operating logic of a line-following robot using **two sensors**. Based on the possible sensor output combinations, we constructed **K-maps** to simplify the control logic and determine the required motor control signals. This helped us to clearly understand how the robot should respond to different line positions. After gaining a rough understanding of the system behavior using two sensors, we extended the same approach to a **three-sensor** configuration in order to improve line detection accuracy and control reliability. The corresponding K-maps and logic diagrams were drawn and are included below.

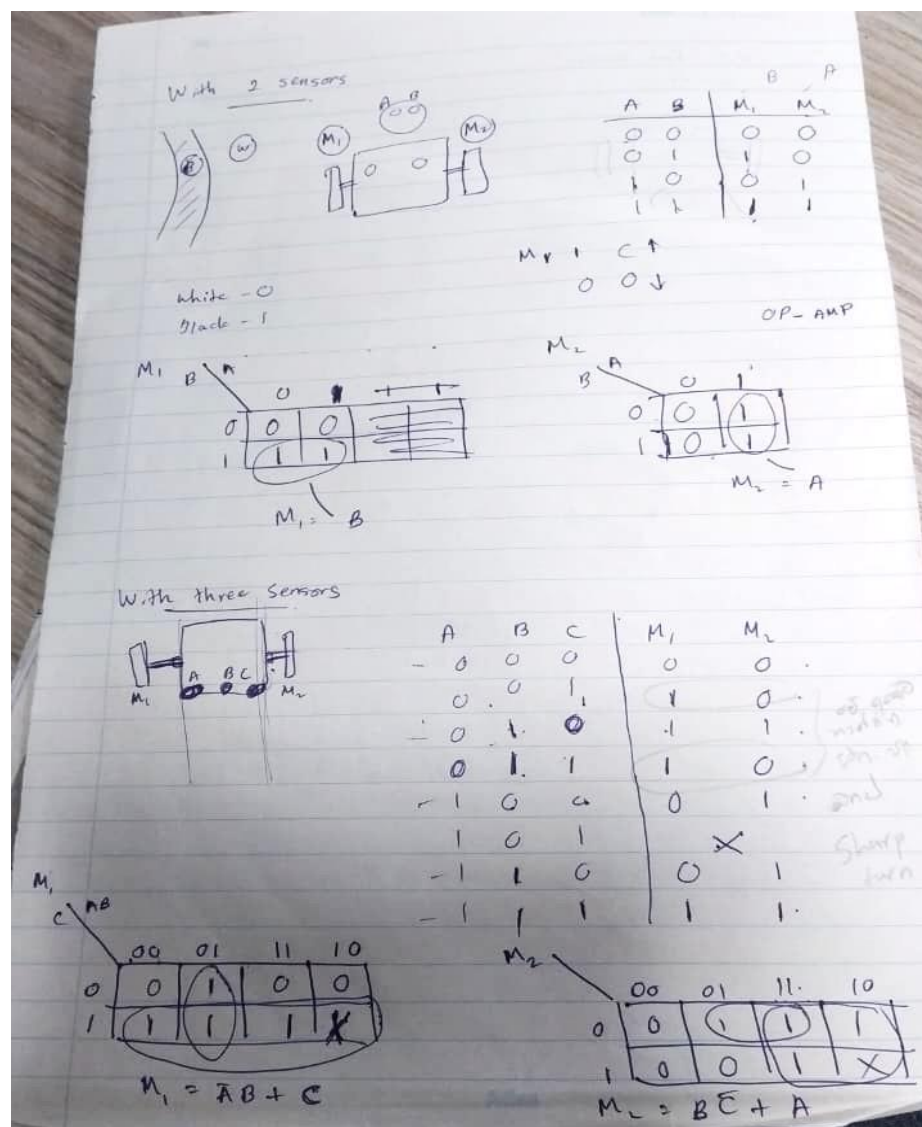


Figure 1: Karnaugh map representation for sensor-based motor control logic

Week 02

Week 2 – Sensor Signal Conditioning and Testing

In Week 2, we discussed methods to convert the **analog output** of the line sensors **into digital signals** suitable for control logic implementation. For this purpose, we decided to use **operational amplifiers with adjustable potentiometers** as comparators to set appropriate threshold levels. A rough circuit diagram was designed for the **TCRT5000** sensor interfaced with an op-amp, and different sensor array configurations were evaluated to improve line detection accuracy. Through this analysis, we identified that a **circular or V-shaped** sensor array provides better line tracking performance compared to a straight-line configuration. Following the discussion session, the designed circuit was physically implemented and tested, using LED indicators to verify high and low output states of the comparator. The related circuit diagrams, implemented hardware, and sensor array configurations are shown below.

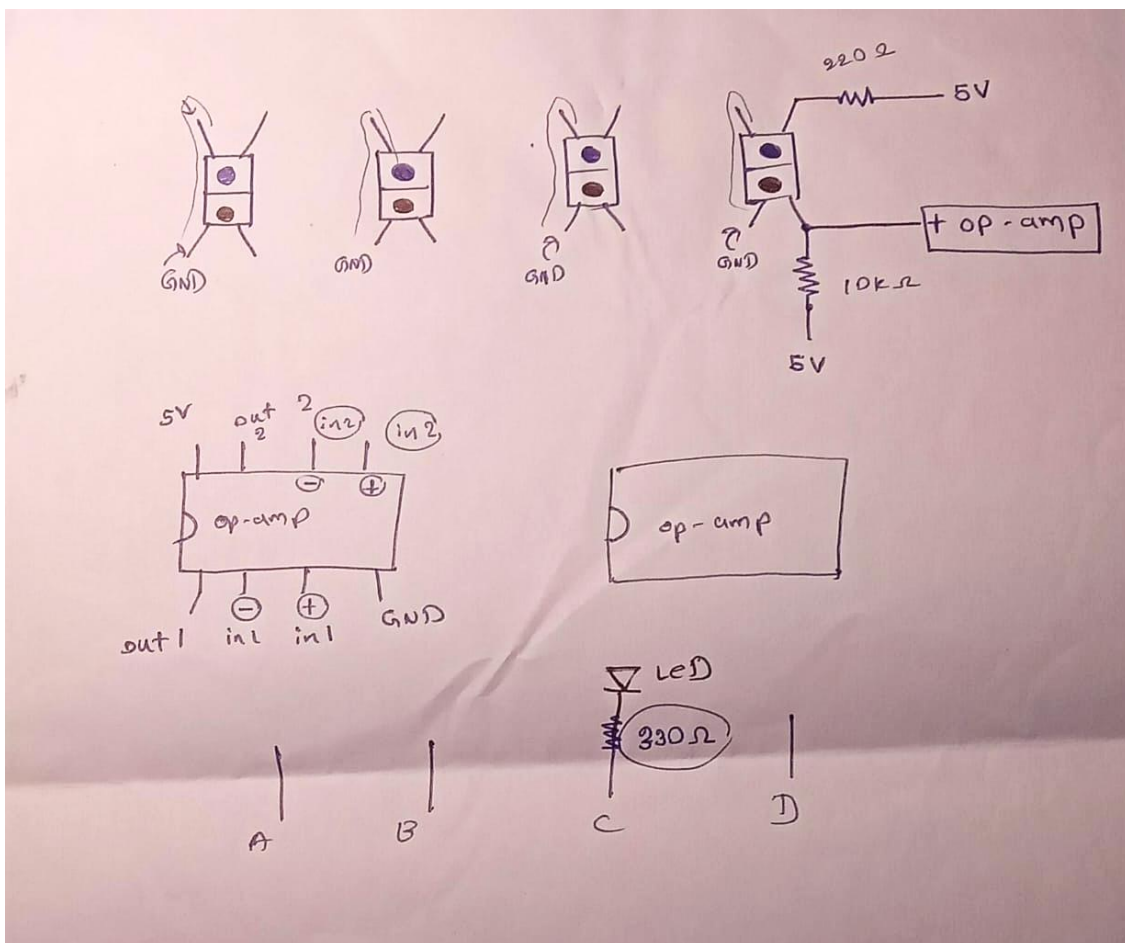


Figure 2.1: Rough circuit diagram of TCRT5000 sensor with op-amp comparator(LM358)

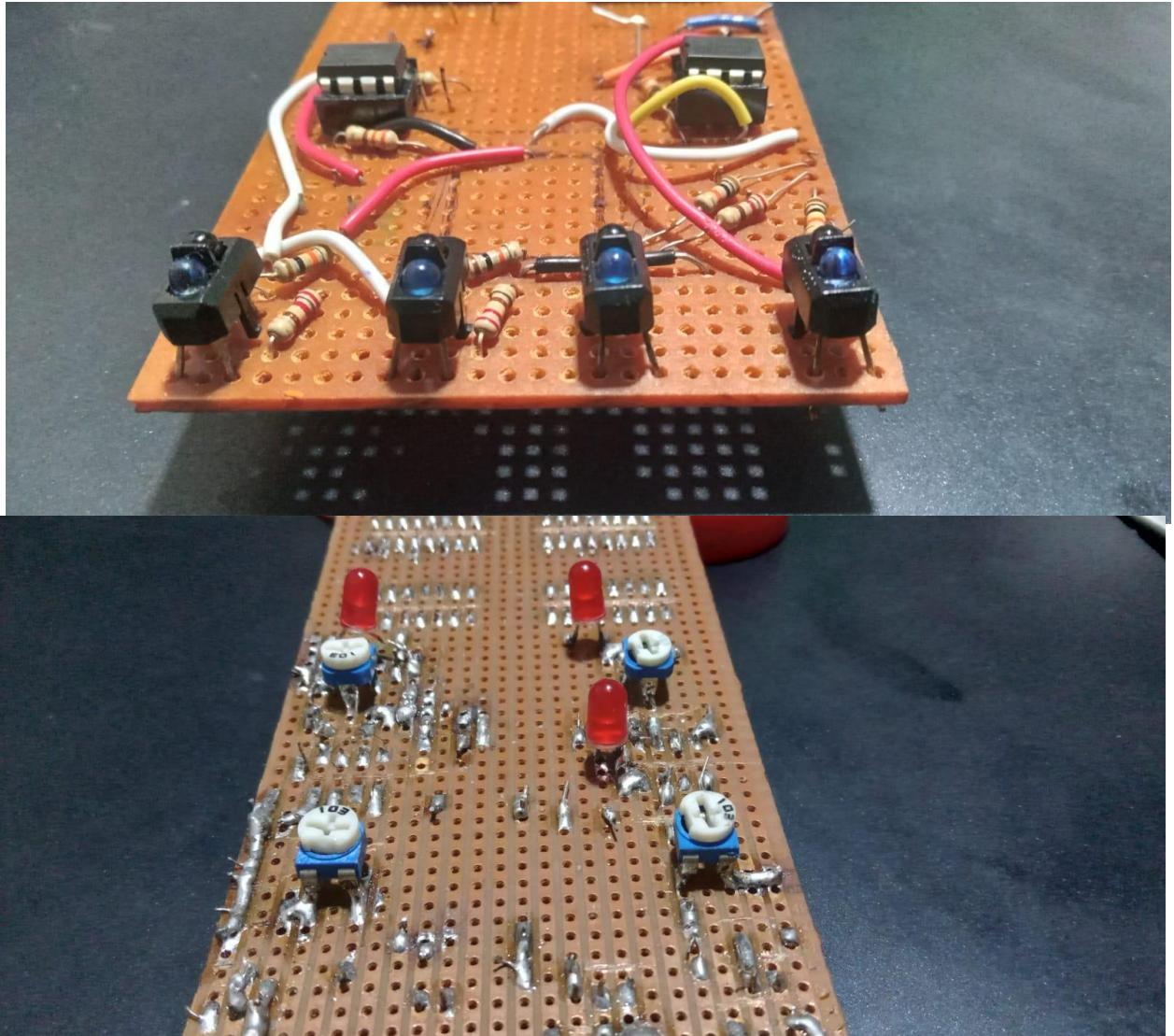


Figure 2.2: Physically implemented sensor and comparator circuit

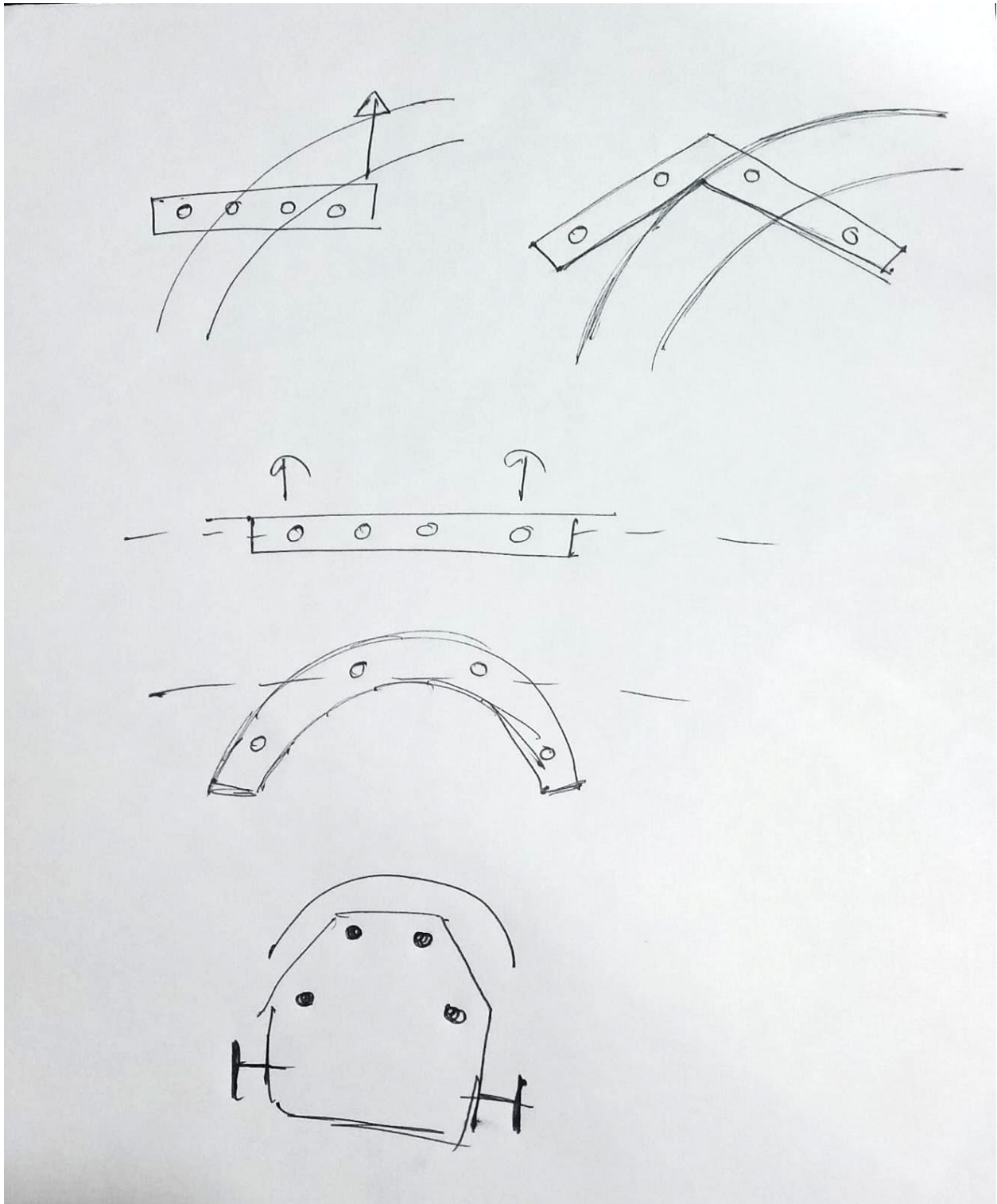


Figure 2.3: Comparison of different sensor array configurations

TCRT5000 Reflective Optical Sensor

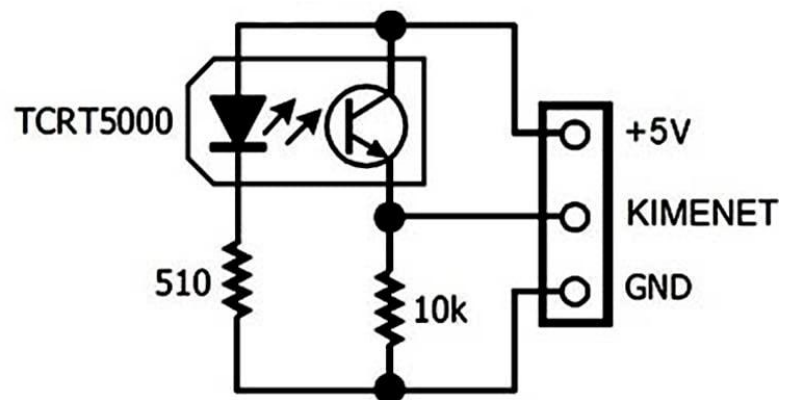
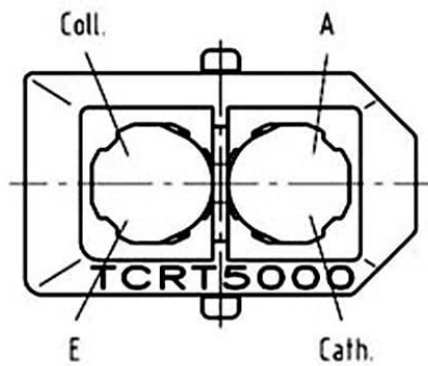
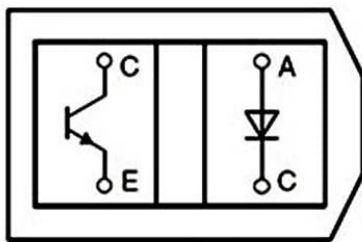


Figure 2.4: Pin configuration of the TCRT5000 reflective IR sensor (source: [click here](#))

PINOUT LM358

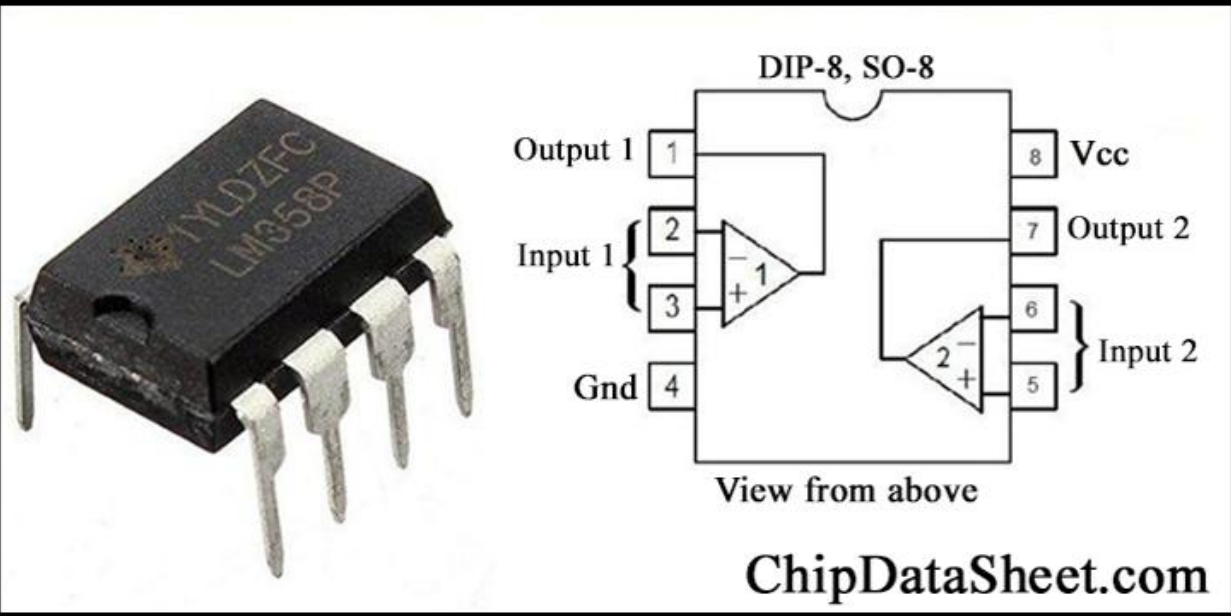


Figure 2.5: Pin configuration of the LM358 IC (source:[click here](#))

Week 03

Week 3 – Multi-Sensor Logic Design and Speed Control

In Week 3, we improved the line follower design by extending the system from **three sensors to four sensors**. Based on the different line curvature conditions, **Karnaugh maps were constructed** to derive the control logic, and the corresponding **IN1, IN2, IN3, and IN4 inputs of the motor driver** were determined. The control strategy was designed such that, for **slight turns**, one wheel is driven while the other remains stopped, whereas for **sharp turns**, the wheels are driven in **opposite directions** to achieve faster reorientation.

During the discussion session of the same week, we explored methods to apply **different PWM signals** to the motor driver in order to control motor speed. As a fully analog solution, the **NE555 timer IC** was selected as a PWM generator. In addition, we discussed how the four-sensor logic could be extended to a **six-sensor configuration**. It was observed that by combining the outermost sensors on both sides using **OR gates**, the six-sensor system can be effectively reduced to an equivalent four-sensor input, allowing the previously designed logic to be reused. The derived logic diagrams, sensor-reduction method diagrams are included below.

.

$M_1(F) [2n1]$

AB \ CD	00	01	11	10
00	0	X	0	0
01	1	X	X	X
11	1	X	1	X
10	X	1	X	X

$$Z = C + D$$

$M_2(F) [2n3]$

AB \ CD	00	01	11	10
00	0	X	1	1
01	0	X	X	X
11	0	X	1	X
10	X	1	X	X

$$Z = A + B$$

$M_3(B) [2n2]$

AB \ CD	00	01	11	10
00	0	X	0	1
01	0	X	X	X
11	0	X	0	X
10	X	0	X	X

$$Z = A\bar{B}$$

$M_4(B) [2n4]$

AB \ CD	00	01	11	10
00	0	X	0	0
01	1	X	X	X
11	0	X	0	X
10	X	0	X	X

$$Z = \bar{C}D$$

E_A

AB \ CD	00	01	11	10
00	0	0	0	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$$C + D + A\bar{B}$$

E_B

AB \ CD	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	0	1	1	1
10	0	1	1	1

$$A + B + \bar{C}D$$

Figure 3.1: Four-sensor logic design and Karnaugh map simplification

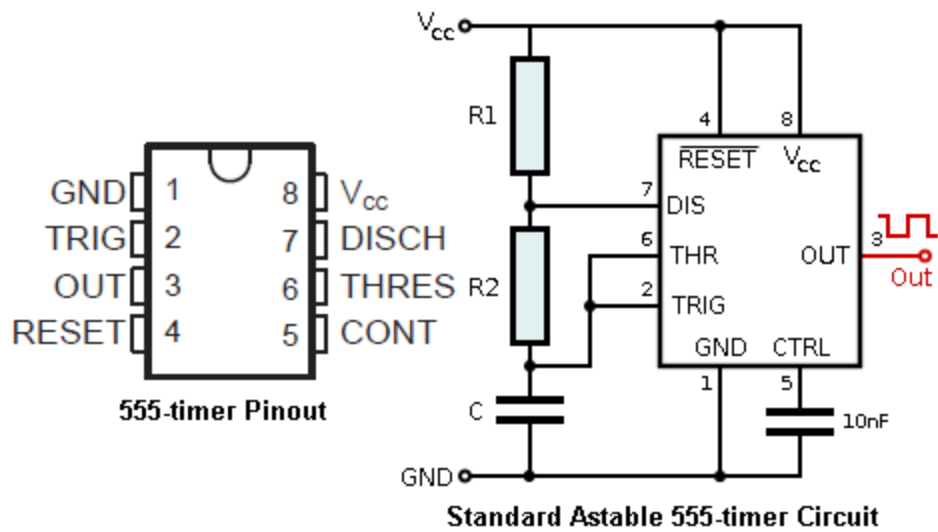


Figure 3.2: NE555 timer IC (Source: [click here](#))

Diagram of a six-sensor configuration reduced to four sensors. The diagram shows a grid of sensors (S₁ to S₆) and a table of their states (0 or 1) for two different configurations (M_L and M_R). The sensors are labeled S₁ to S₆ and are connected to a common bus. The table shows the states of the sensors for two different configurations (M_L and M_R). The sensors are labeled S₁ to S₆ and are connected to a common bus. The table shows the states of the sensors for two different configurations (M_L and M_R).

Sensors						Configurations			
S ₁	S ₂	S ₃	S ₄	S ₅	S ₆	M _L		M _R	
S ₁	S ₂	S ₃	S ₄	S ₅	S ₆	F	B	F	B
0	0	0	0	0	1	1	0	0	1
0	0	0	0	1	1	1	0	0	1
0	0	0	1	1	0	1	0	0	0
0	0	1	1	0	0	1	0	1	0
0	1	1	0	0	0	0	0	1	0
1	1	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	1	1	0
0	0	0	0	0	0	0	0	0	0
0	0	1	1	1	0	1	0	0	0
0	1	1	1	0	0	0	0	1	0
0	0	0	1	1	0	1	0	0	1
1	1	1	0	0	0	0	1	1	0
0	1	1	1	1	0	1	0	1	0
1	1	1	1	1	0	1	0	1	0
0	0	1	1	1	1	1	0	0	1

Legend: Black - 1, White - 0

Diagram of a six-sensor configuration reduced to four sensors. The diagram shows a grid of sensors (S₁ to S₆) and a table of their states (0 or 1) for two different configurations (M_L and M_R). The sensors are labeled S₁ to S₆ and are connected to a common bus. The table shows the states of the sensors for two different configurations (M_L and M_R).

Figure 3.3: Six-sensor configuration reduced to four sensors

Week 04

Week 4 (02.01 - 02.08) – Implementation of D-Latch Based Memory Logic

In Week 05, we focused on improving the two-sensor line follower by incorporating memory elements using **D-latches**. The objective was to derive suitable Boolean expressions for two D-latches in order to enhance the robot's response when it deviates from a curve.

Logic expressions were developed such that when the robot moves away from the line during a turning condition, the stored state in the D-latch enables the system to guide the robot back toward the line instead of stopping immediately. This approach improves stability and reduces unnecessary interruptions in motion.

The complete derivation of the D-latch logic expressions and corresponding circuit diagrams are included below.

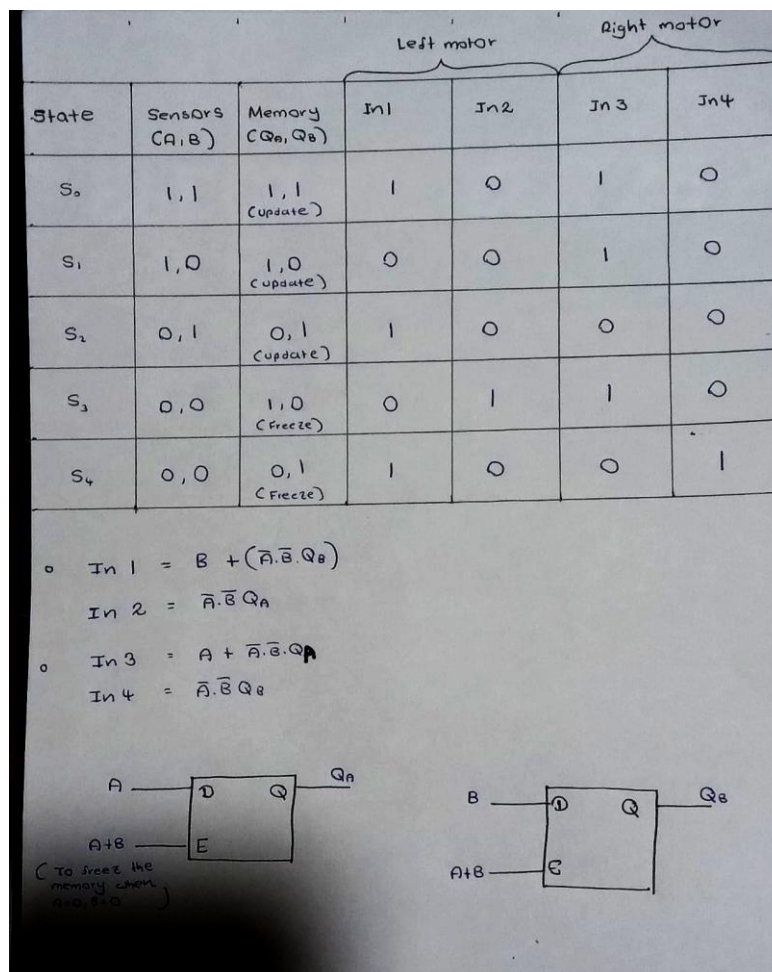


Figure 4.1: Derivation of Boolean expressions for D-latch based control logic

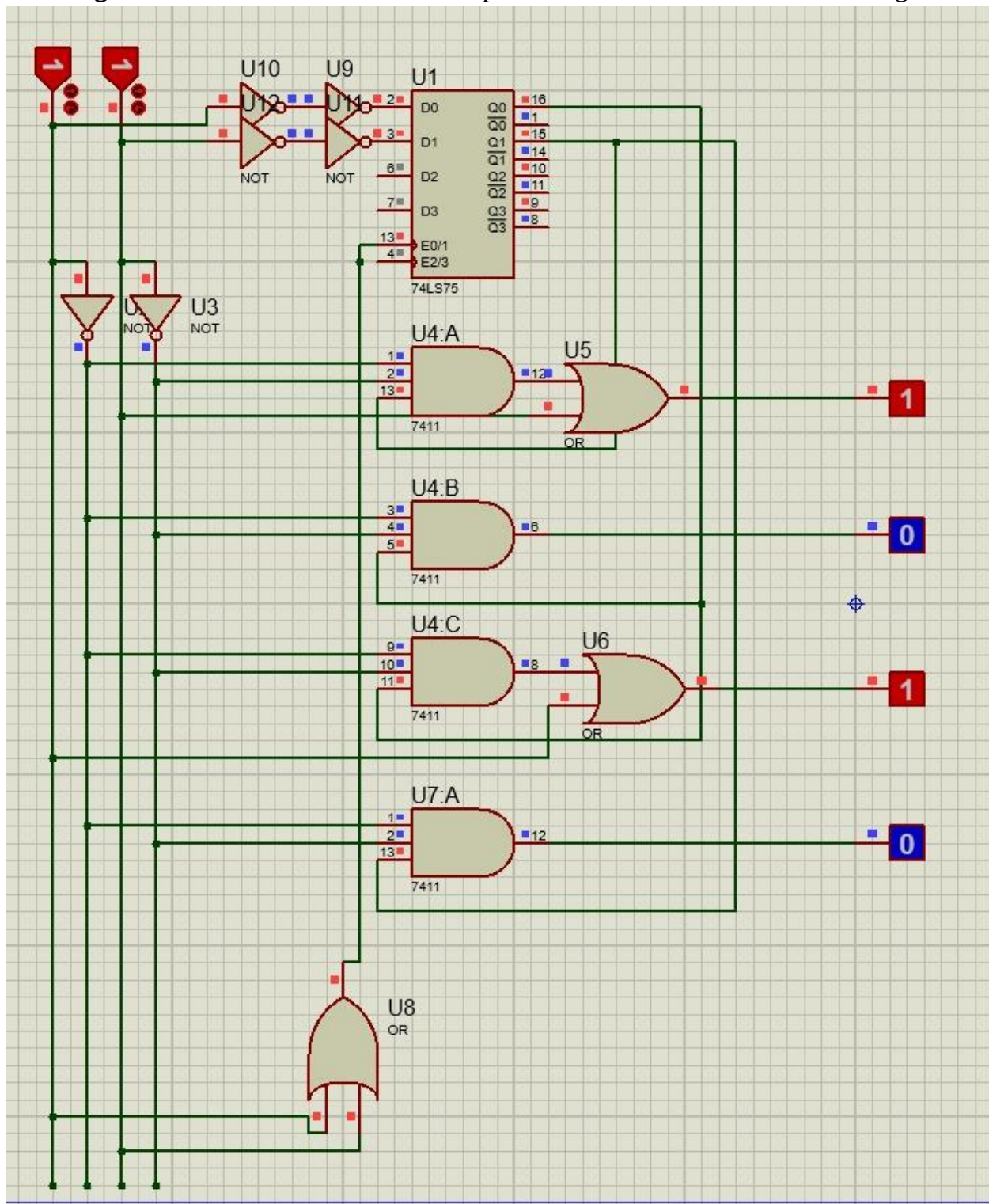


Figure 4.2: D-Latch Based Circuit in Proteus

Week 05

Week 5 (02.08 – 02.15) – Prototype Implementation and Testing

During this week, a functional test prototype of the line-following robot was constructed using **two KY-033 line tracking sensors**, an **L293D motor driver IC**, and **two N20 200 RPM (6 V) DC motors**. The system was powered by **two 3.7 V, 1850 mAh Li-ion batteries connected in series**, providing 7.4 V, and **LM2596 DC buck converter** was used to step down the voltage to 5 V for appropriate circuit operation.

The control logic implemented in this prototype was based on the Boolean expressions derived during the Week 1 discussion session for the two-sensor configuration. After assembling the hardware, the robot was tested on a **black tape line track** to evaluate its performance. The prototype successfully followed the line according to the designed logic. Images of the constructed robot and a video recording of the testing procedure are included below for reference.

Video 1: Two-Sensor Line Follower Testing

[Click here to view the testing video](#)

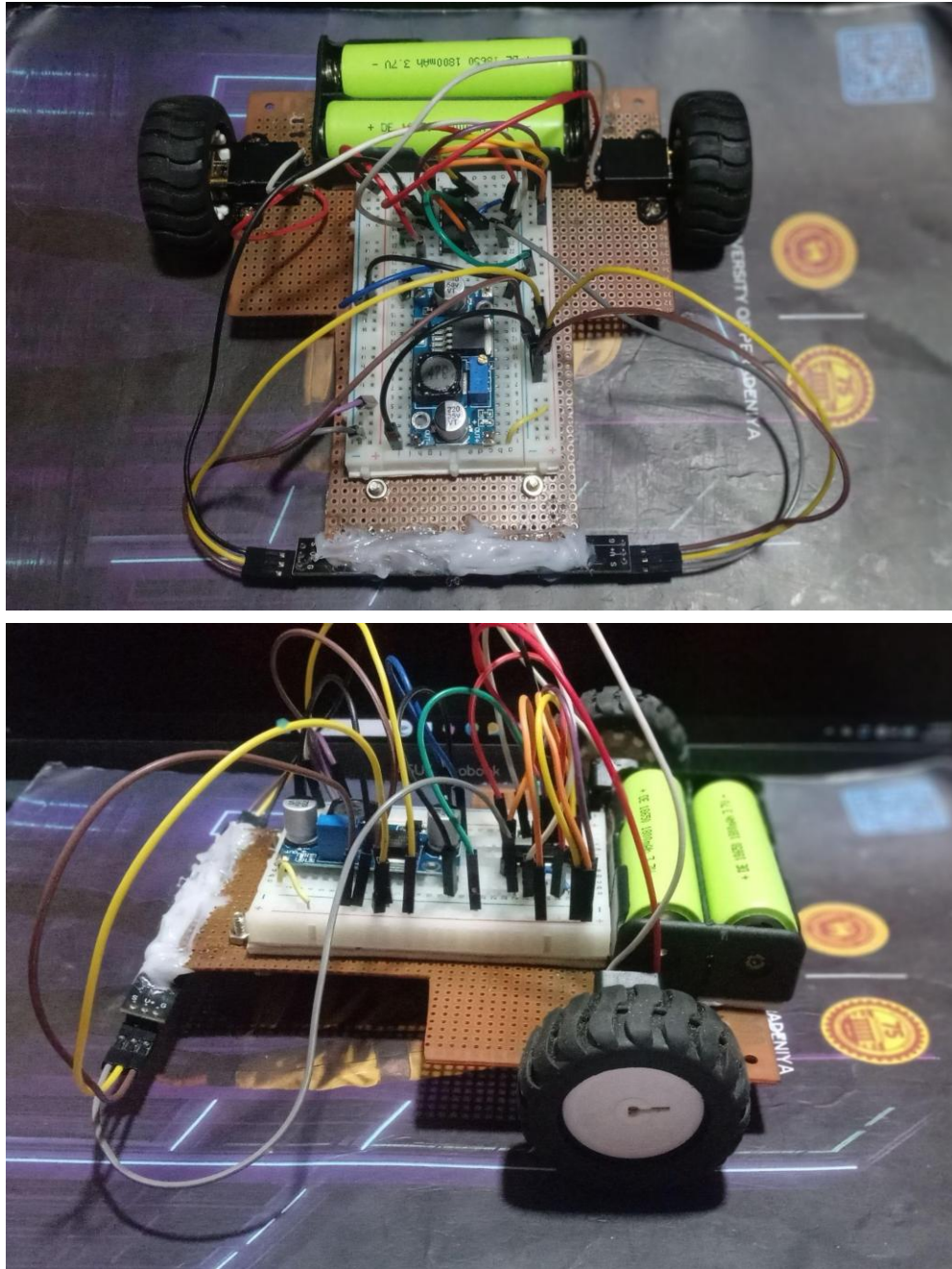


Figure 4.1: Constructed two-sensor line follower prototype

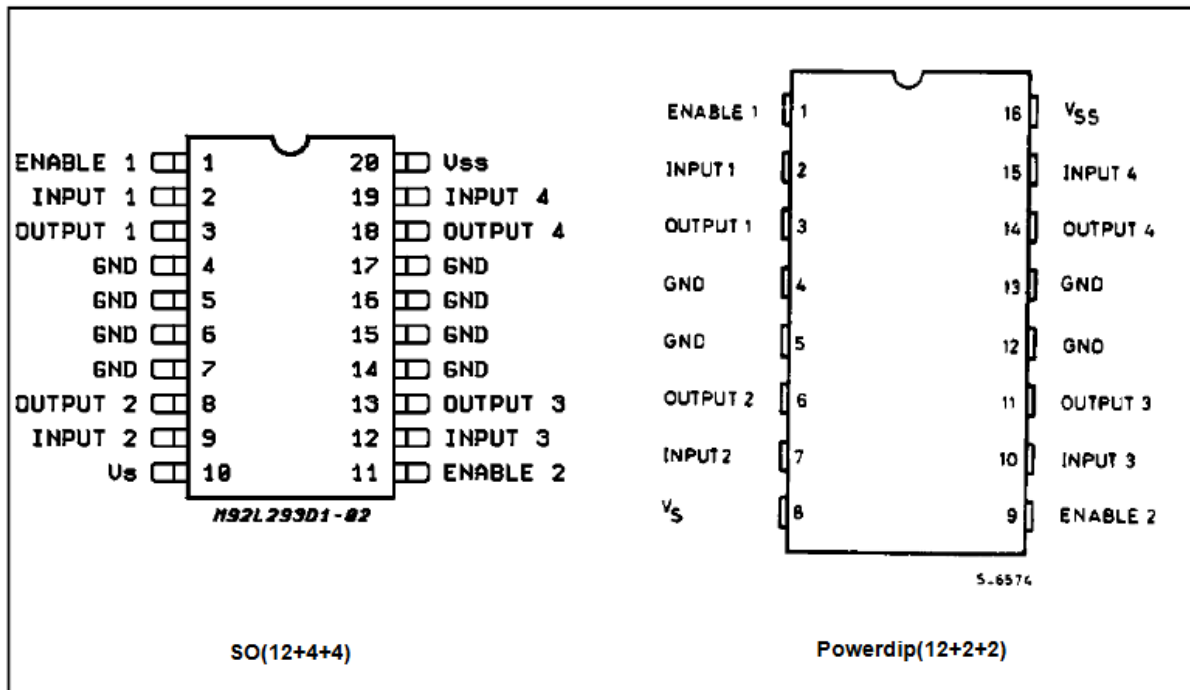


Figure 4.2: Pin configuration of the L293D IC (Source:[click here](#))

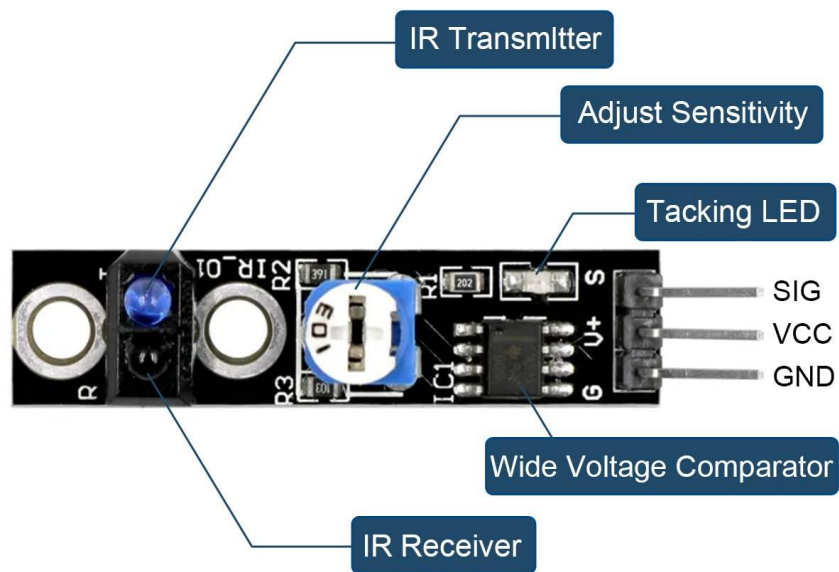


Figure 4.3: Pin configuration of the KY-033 sensor (Source:[click here](#))

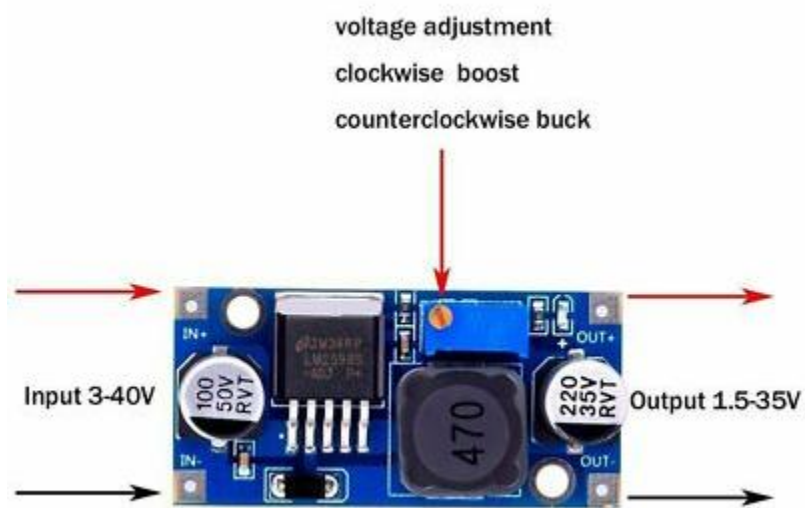


Figure 4.4: Pin configuration of the LM2596 DC buck converter (Source: [click here](#))

Week 06

Week 6 (02.15 – 02.22) – Testing and Identified Limitations

In Week 6, the two-sensor line follower prototype was tested on the **final project track**. The robot performance exceeded our initial expectations, demonstrating stable and accurate line tracking under standard conditions.

However, during testing on the dashed-line sections of the track, a limitation was observed. Although we initially expected the robot to continue moving forward due to its operating speed when encountering short white gaps, it instead stopped whenever both sensors detected a white region. This behavior occurred because the existing logic interprets the white space as a loss of line, resulting in a stop condition.

To address this issue, we proposed modifying the control logic such that when both sensors detect white simultaneously for a short duration, the robot continues moving forward instead of stopping. This adjustment is expected to improve performance over dashed-line segments while maintaining stability in normal operation.

The testing process and observed behavior are demonstrated in the video provided below.

Video 2: Two-Sensor Line Follower Testing

[Click here to view the testing video](#)

Week 07

Week 07 (02.22-02.28) – Dash-Line Logic Improvement and NAND-Based Optimization

In Week 07, we addressed the stopping issue observed when the robot encountered dashed-line segments. Previously, when both sensors detected a white surface (logic 0,0), the robot was designed to stop. To overcome this limitation, a new control logic was defined such that when both sensors detect white simultaneously, the robot continues moving forward instead of stopping.

The initial implementation of this modified logic required two OR gates and two NOT gates, which resulted in the need for two additional ICs. However, further simplification was performed by redesigning the logic using only NAND gates. Through NAND-only implementation, the same functionality was achieved using four NAND gates, requiring only one additional IC. This significantly reduced hardware complexity and improved circuit efficiency.

The updated logic derivation, NAND-based circuit implementation, and dash-line performance testing are presented below.

Video 3: Performance testing on dashed-line track after logic modification

[Click here to watch the video](#)

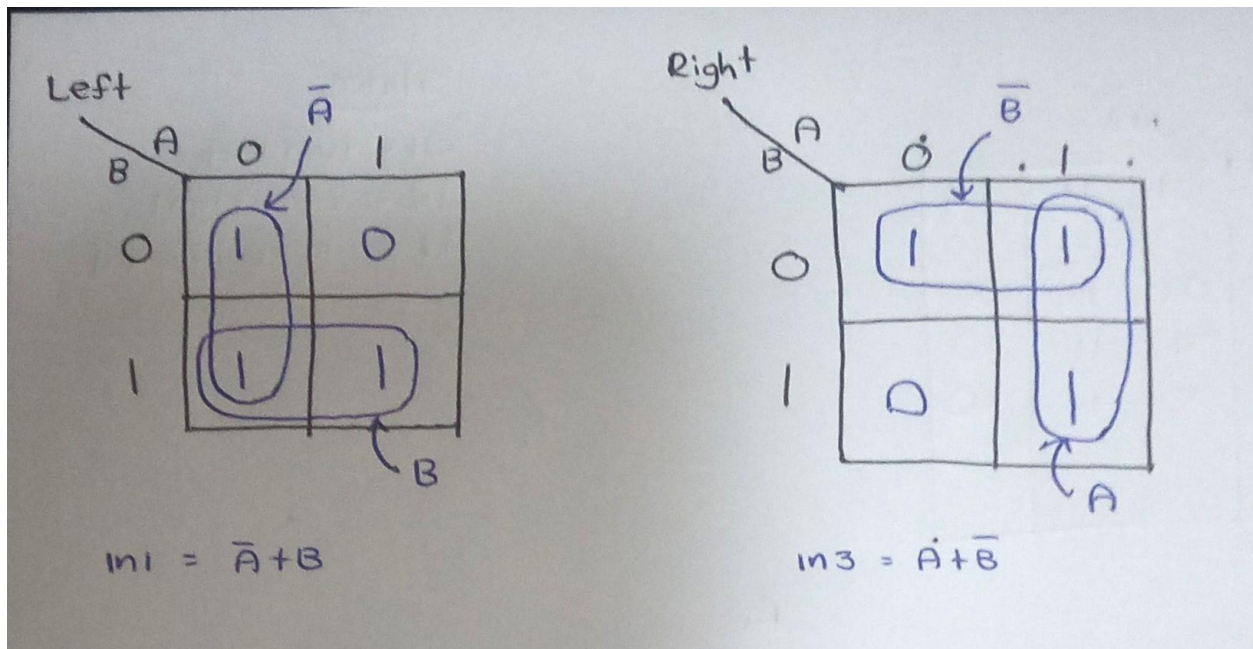


Figure 7.1: Modified control logic derivation for dash-line handling

* NAND only

$$\begin{aligned}
 m1 &= \overline{\overline{\bar{A} + B}} \\
 &= \overline{\bar{A} \cdot \bar{B}} \\
 &= \overline{\bar{A} \cdot \bar{B}} //
 \end{aligned}$$

$$\begin{aligned}
 m3 &= \overline{\overline{A + \bar{B}}} \\
 &= \overline{\bar{A} \cdot B} \\
 &= \overline{\bar{A} \cdot B} //
 \end{aligned}$$

Figure 7.2: NAND-only implementation of improved control logic

Week 08

Week 08 (03.08 - 03.15) – Final Track Testing and Analysis of Dash-Line Deviation

In Week 08, the robot was tested again on the **final track** to evaluate the effectiveness of the previously implemented improvements. During this test, it was observed that the robot deviates from the track when encountering **dashed-line segments**, particularly while negotiating curves.

Upon analysis, we identified that during a turning condition, **both sensors briefly pass over the first white gap** in the dashed line. This causes the control logic to misinterpret the situation, resulting in the robot moving away from the intended path.

As a preliminary solution, we proposed **reducing the length of the robot chassis**, which would allow the sensors to follow the curve more closely. This adjustment is expected to minimize simultaneous white detection during turns and improve performance over dashed-line sections.

The observed behavior, along with a conceptual explanation of the issue and the proposed solution, are illustrated in the video and sketch provided below.

Video 4: Robot deviation observed on dashed-line curve in final track

[click here to watch the video](#)

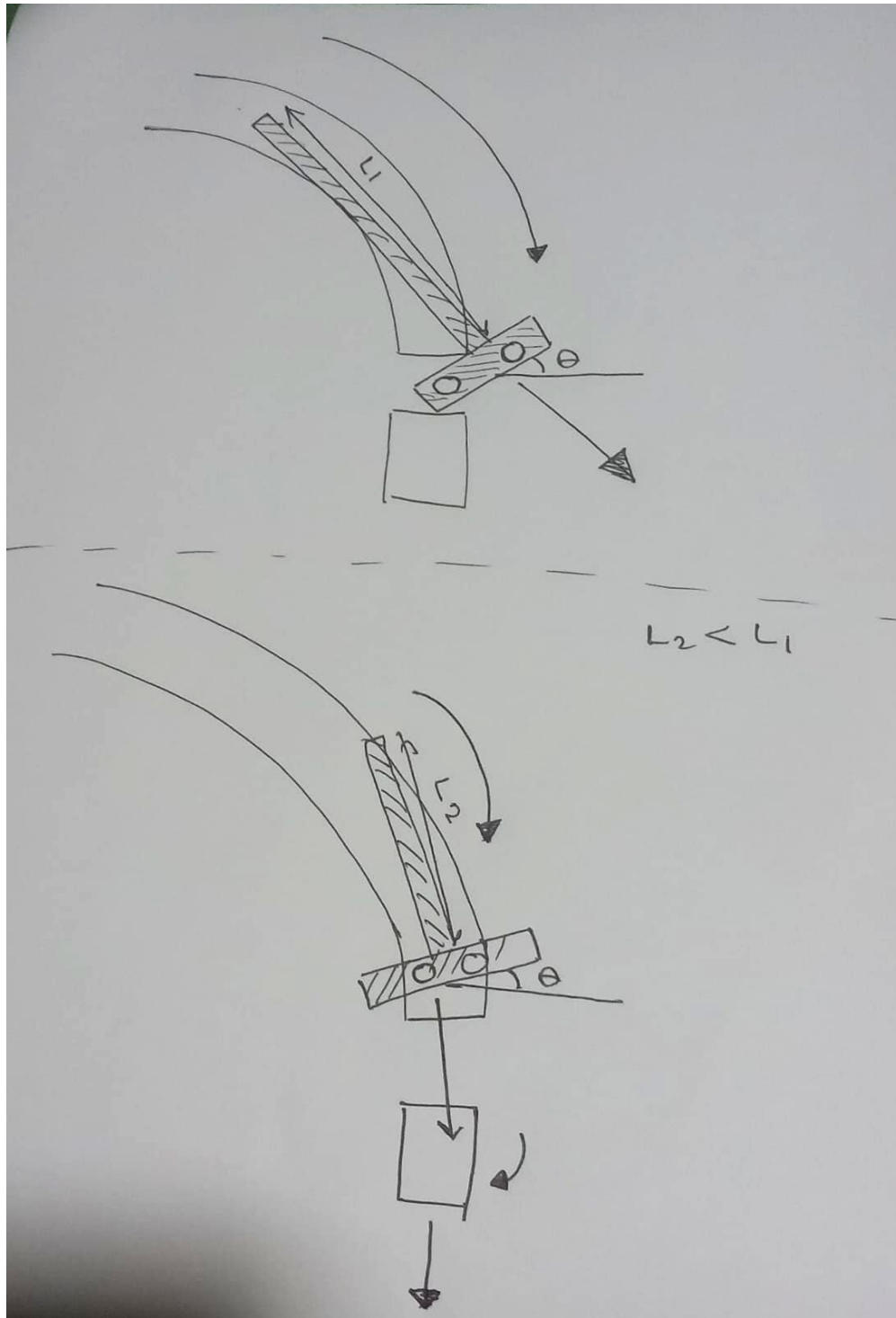


Figure 8.1: Sketch illustrating sensor behavior during dashed-line curve deviation

Week 09

Week 09 (03.15 - 03.22) – Mechanical Optimization and Final Validation

In Week 09, the physical design of the line follower robot was refined by **reducing the overall chassis length** to improve maneuverability, especially on curved and dashed-line sections. In addition, the circuit implementation was transitioned from a **breadboard with jumper wires to a soldered vero board**, resulting in a more stable and reliable hardware configuration. This change minimized loose connections and improved signal consistency.

Although integrating all components into the smaller chassis was challenging, the compact design contributed to improved tracking performance. After completing these modifications, the robot was tested on the **final track**, where it demonstrated stable and accurate operation. To achieve optimal performance, **fine adjustments were made to the potentiometer settings** of both sensors to ensure correct threshold detection.

The final robot design and its performance on the track are shown in the images and video provided below.

Video 5: Final performance of the robot on the project track

[click here to watch the video](#)

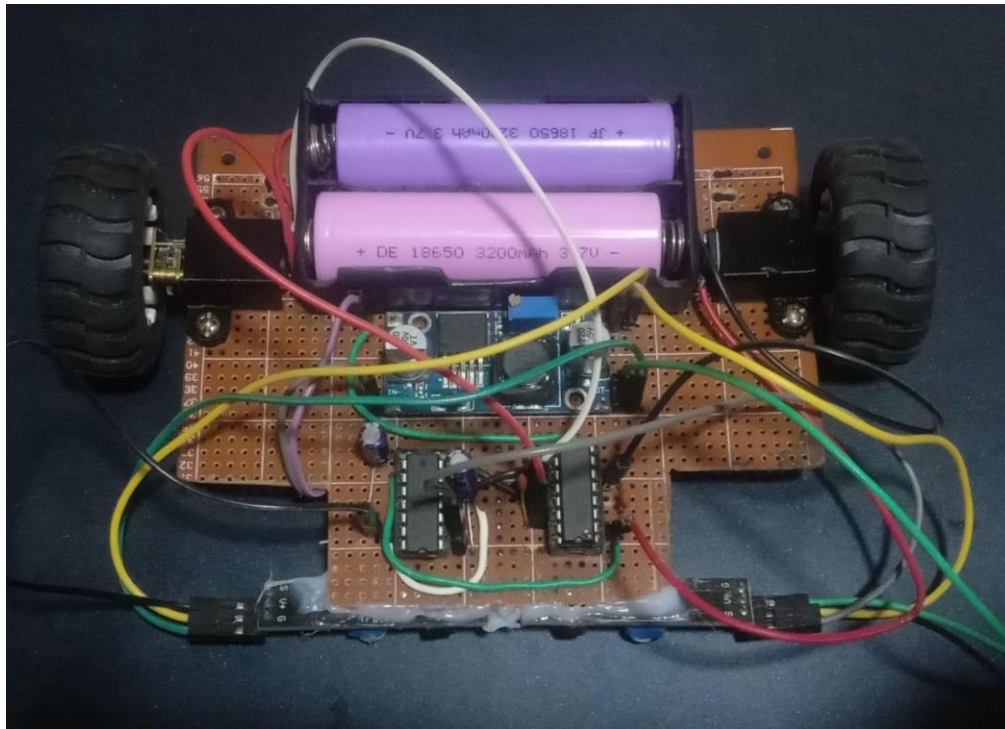
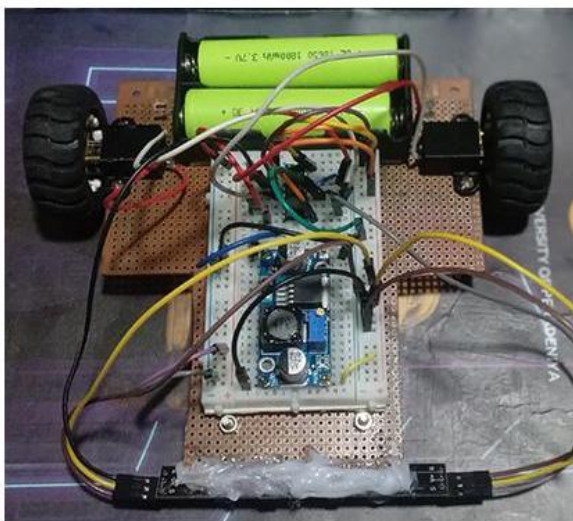


Figure 9.1: Final compact line follower robot implemented on vero board

BEFORE



AFTER

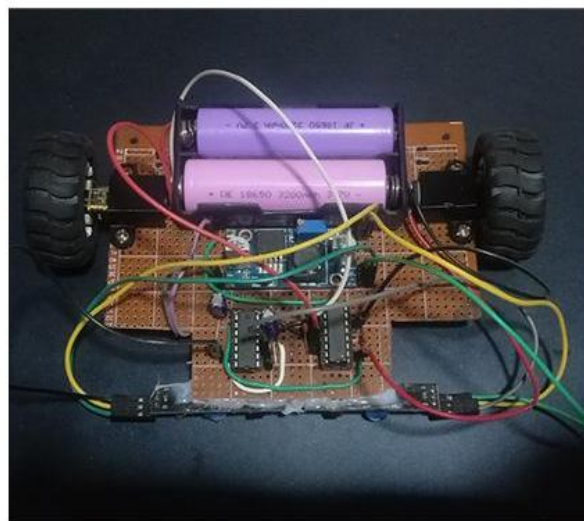


Figure 9.2: Before and After Robot Design

Week 10

Week 10 (04.19 - 04.25) – Flip-Flop Based Design and PCB-Oriented Development

In Week 10, we initiated the design of a new line follower system incorporating **flip-flop based control logic** and a **printed circuit board (PCB)** to achieve a cleaner and more reliable hardware implementation. Unlike the previous prototype, which did not utilize flip-flops and was not PCB-based, this design aims to improve structural organization and signal stability.

As a first step, we derived the **logical expressions for the control system using two SR flip-flops** to introduce memory into the line-following behavior. This approach allows the robot to retain state information and respond more effectively to changes in the track.

The derived logic was then **simulated in Proteus Design Suite** to verify correct operation before proceeding to hardware implementation. The logic derivations and simulation results are presented below.

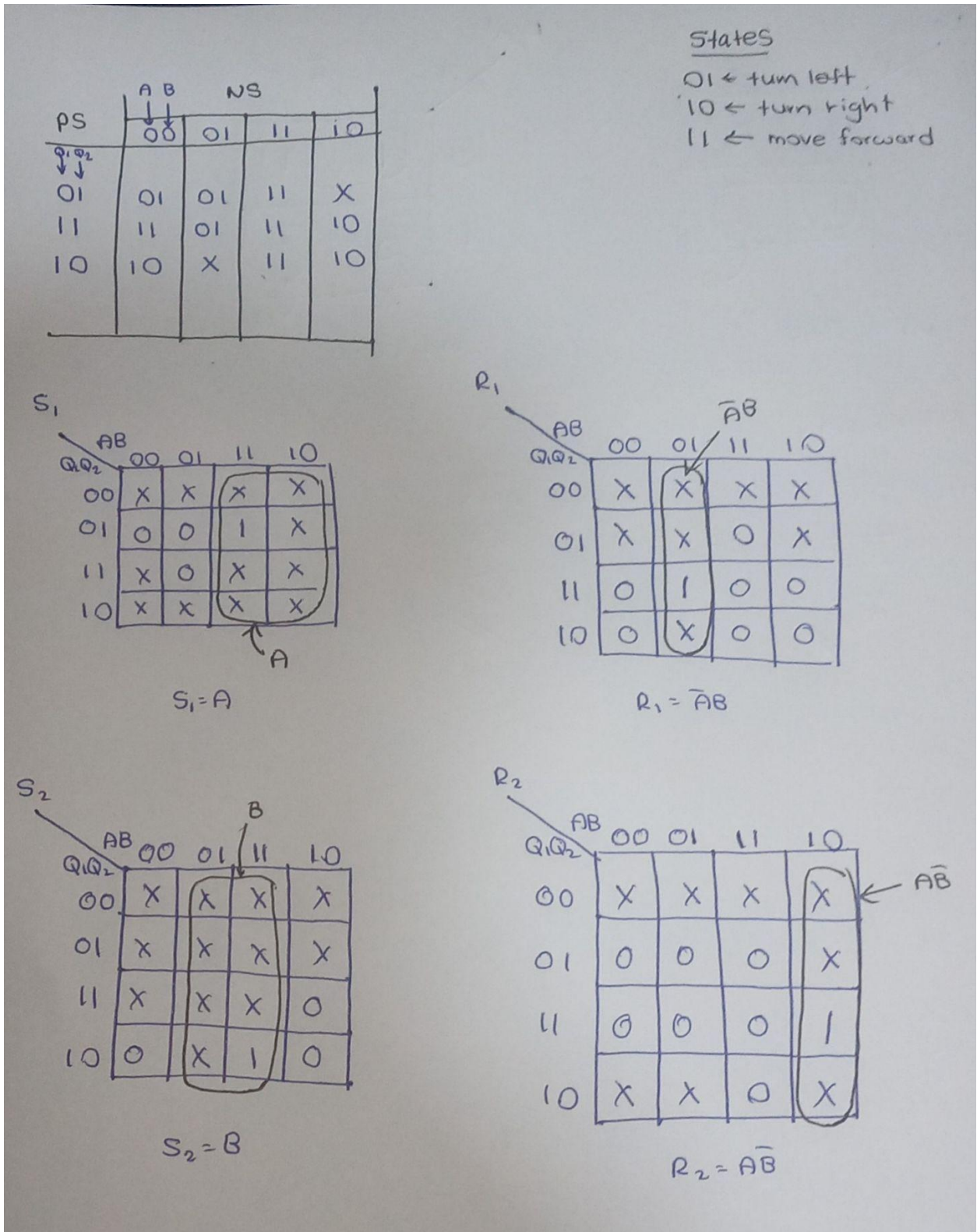


Figure 10.1: Derivation of control logic using SR flip-flops

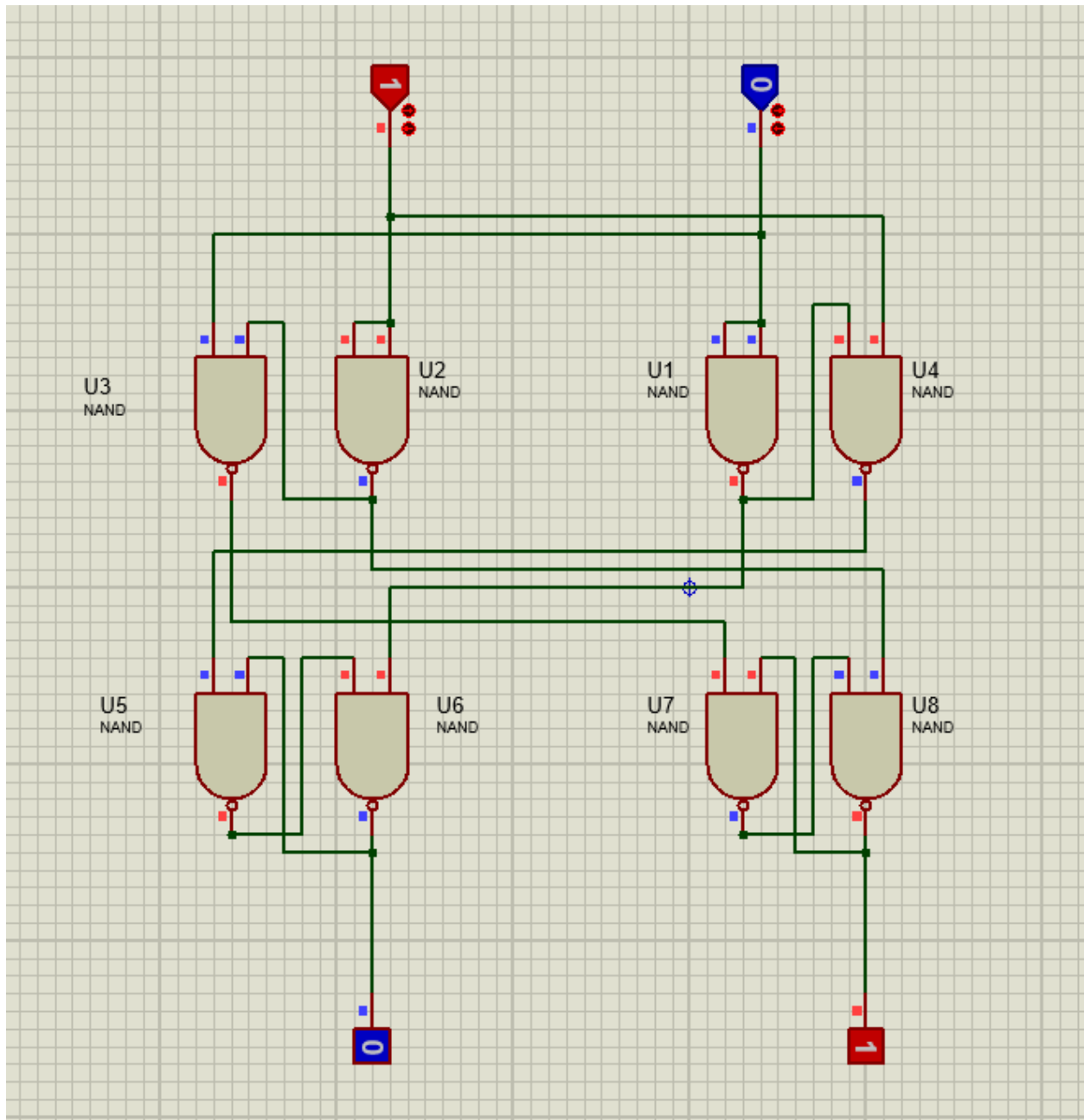


Figure 10.2: Proteus simulation of flip-flop based line follower circuit

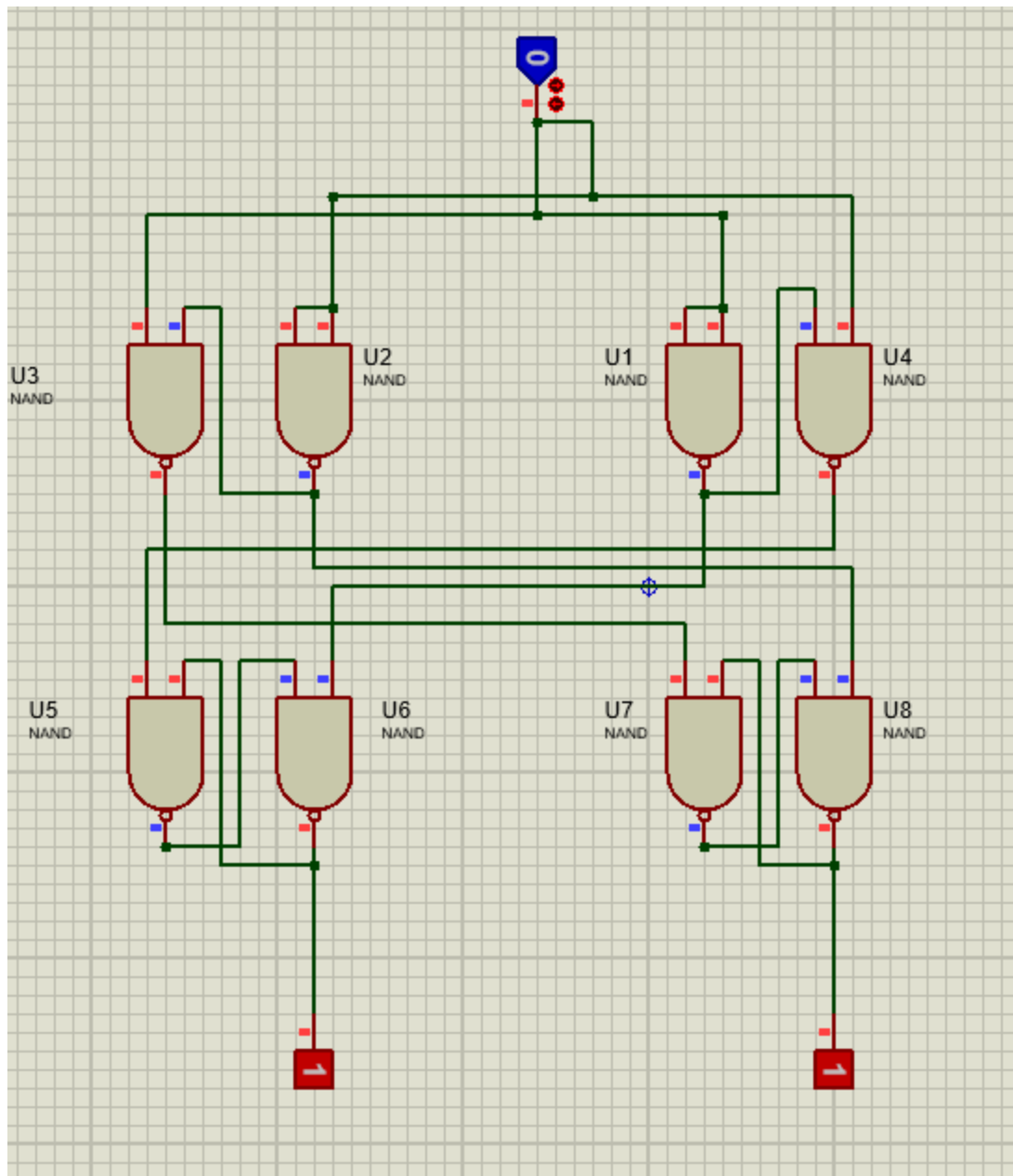


Figure 10.3: Proteus simulation for dash lines

Week 11

Week 11 (04.25 - 05.01) – PCB Fabrication, Assembly, and System Debugging

In Week 11, we designed a **single-layer PCB** for the line follower using **KiCad** and proceeded with in-house fabrication. The PCB layout was printed on **photo paper** and transferred onto a copper board using the **toner transfer (ironing) method**. After transfer, any missing traces were corrected using a **permanent marker**, and the board was then etched in a **Ferric Chloride** solution for approximately 30 minutes with continuous agitation.

Once etching was complete, the remaining toner was removed using **nail polish remover**, and holes were drilled for component placement. The components were then **soldered onto the PCB**, and necessary **air wires** were added due to the single-layer design constraints. Following this, external components including **sensors, motors, wheels, caster wheel, switch, and batteries** were integrated to complete the robot assembly.

During initial power-up, the robot did not function as expected. A systematic debugging process was carried out using a **multimeter continuity test**, which revealed **grounding issues** in the circuit. After correcting and re-soldering the faulty connections, both sensors and motors operated correctly.

Subsequent testing on a track showed unstable behavior. This was resolved by **increasing the distance between the sensors and the ground surface**, which significantly improved detection accuracy. Although a minor **propagation delay** was observed in dashed-line sections due to the use of **SR flip-flops**, it did not critically affect overall performance. An alternative improvement could be the use of **JK flip-flops** to reduce such effects; however, this was not implemented due to the finalized PCB design.

Overall, the robot demonstrated satisfactory performance on the test track and final track after these adjustments. The design process, PCB fabrication stages, assembly, and testing results are illustrated below.

Video 6: Testing of PCB-based robot on track

[click here to watch the video](#)

Video 7: Final performance of the robot on the project track

[click here to watch the video](#)

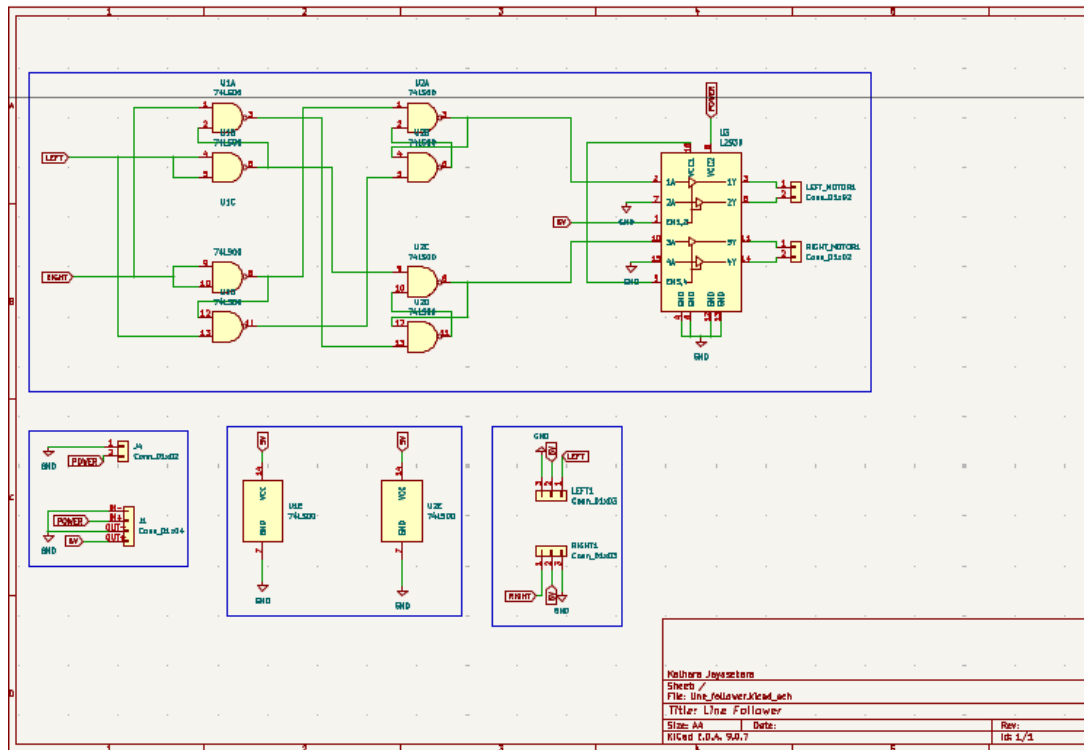


Figure 11.1: Schematic design of the line follower in KiCad

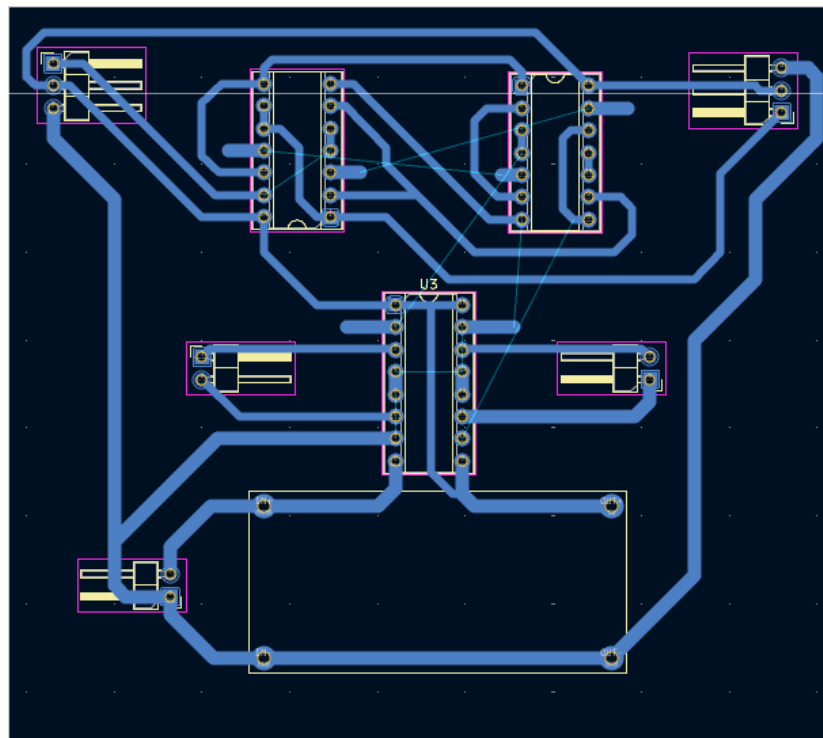


Figure 11.2: PCB layout (single-layer design)

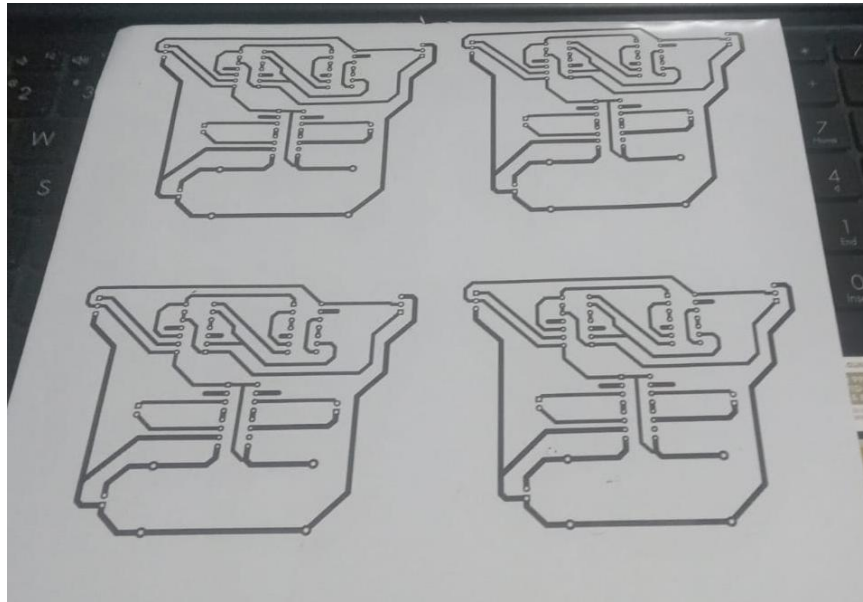


Figure 11.3: Printed PCB layout on photo paper

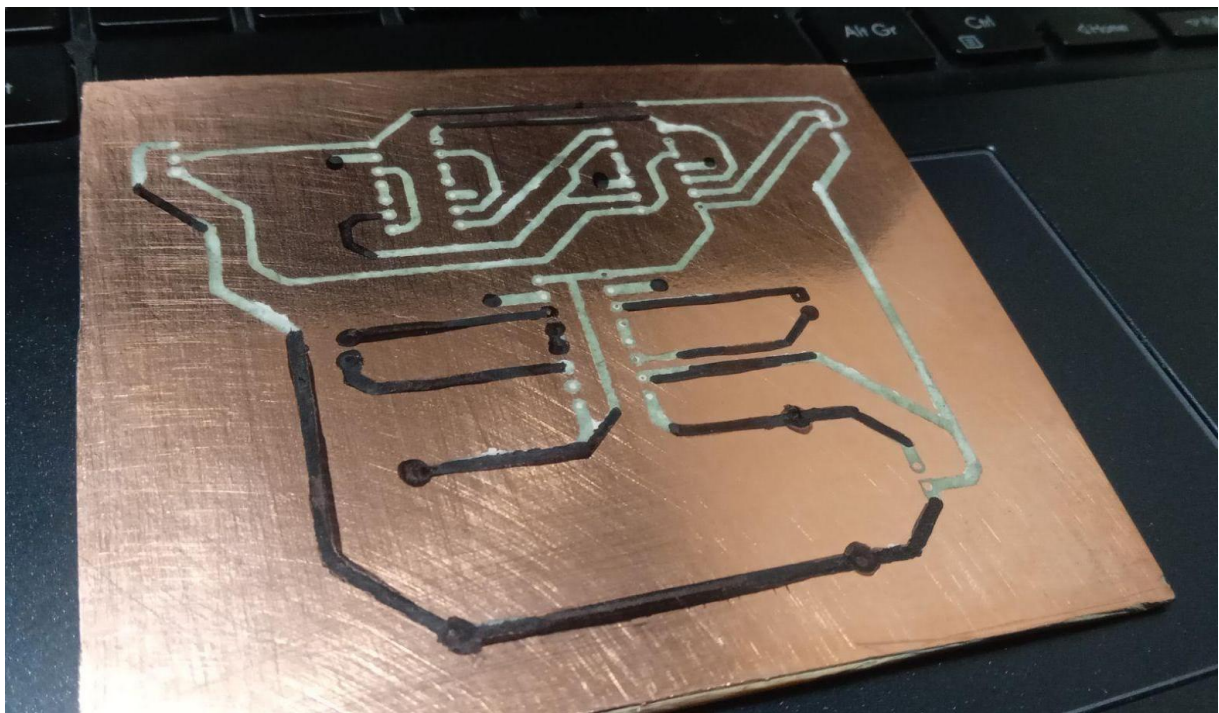


Figure 11.4: PCB after toner transfer onto copper board

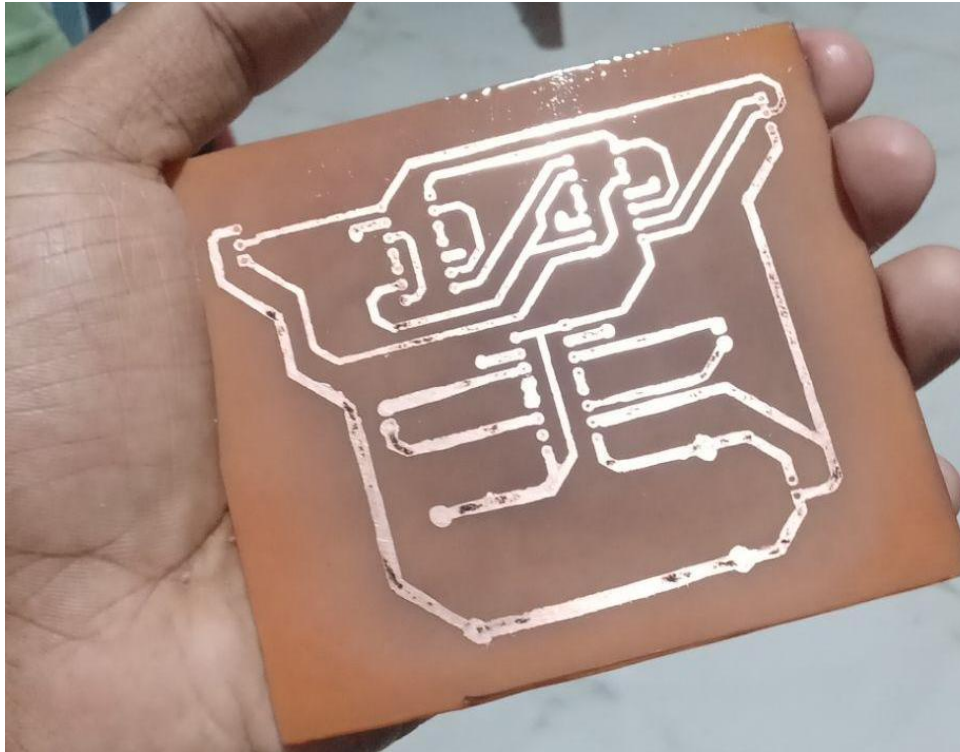


Figure 11.5: Final etched PCB after removing toner

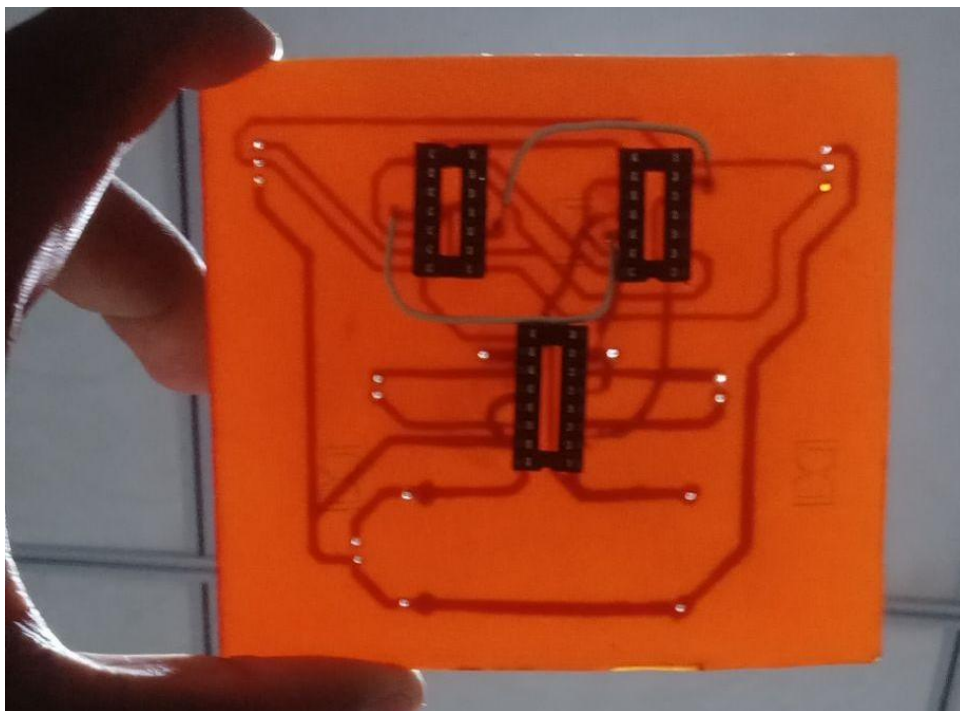


Figure 11.6: Assembled PCB with soldered components and air wires

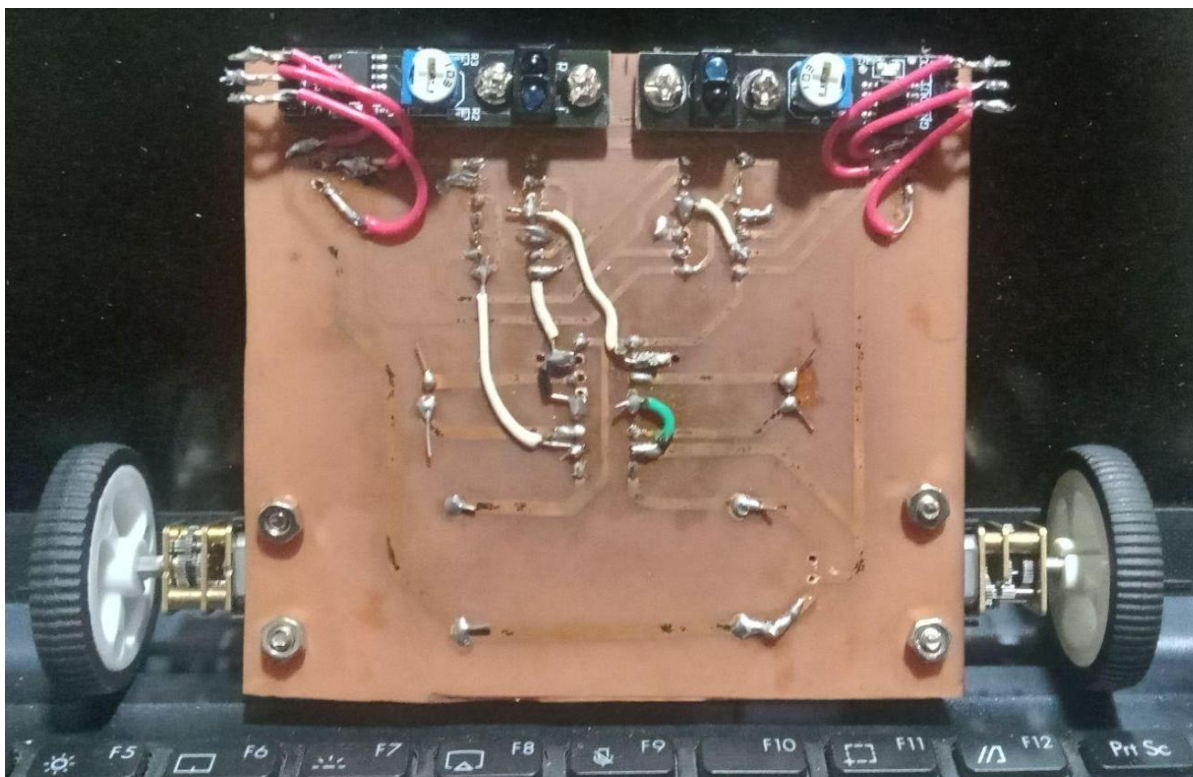
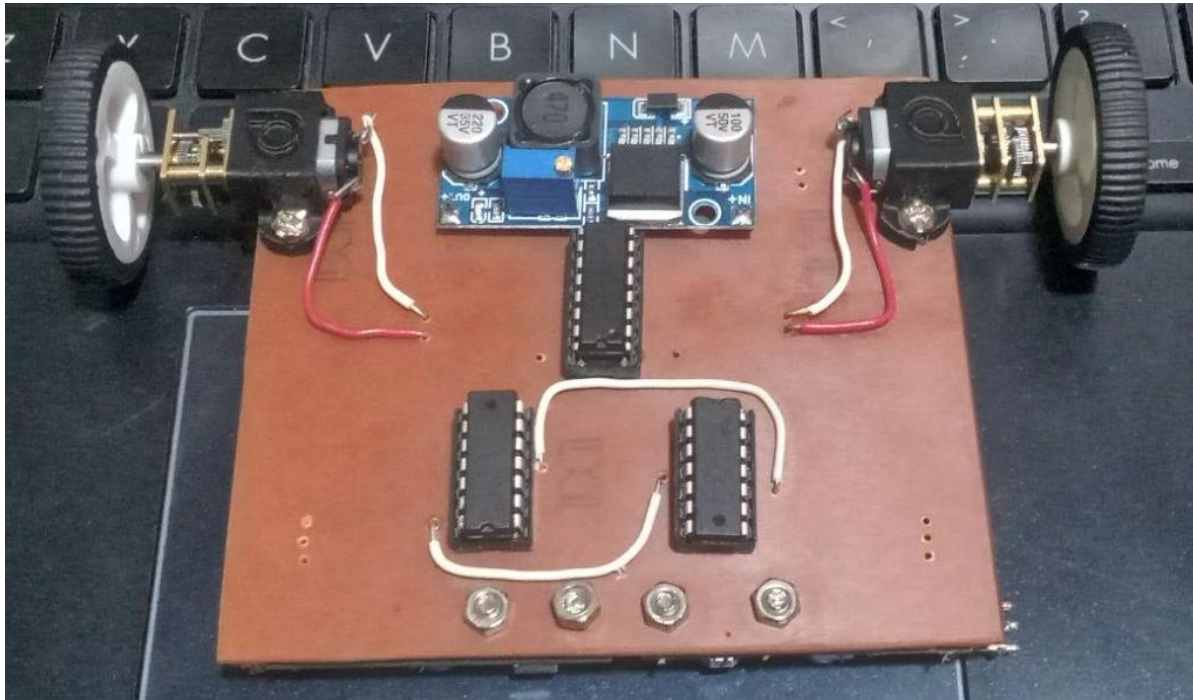


Figure 11.7: Integration of motors, sensors, and control circuitry

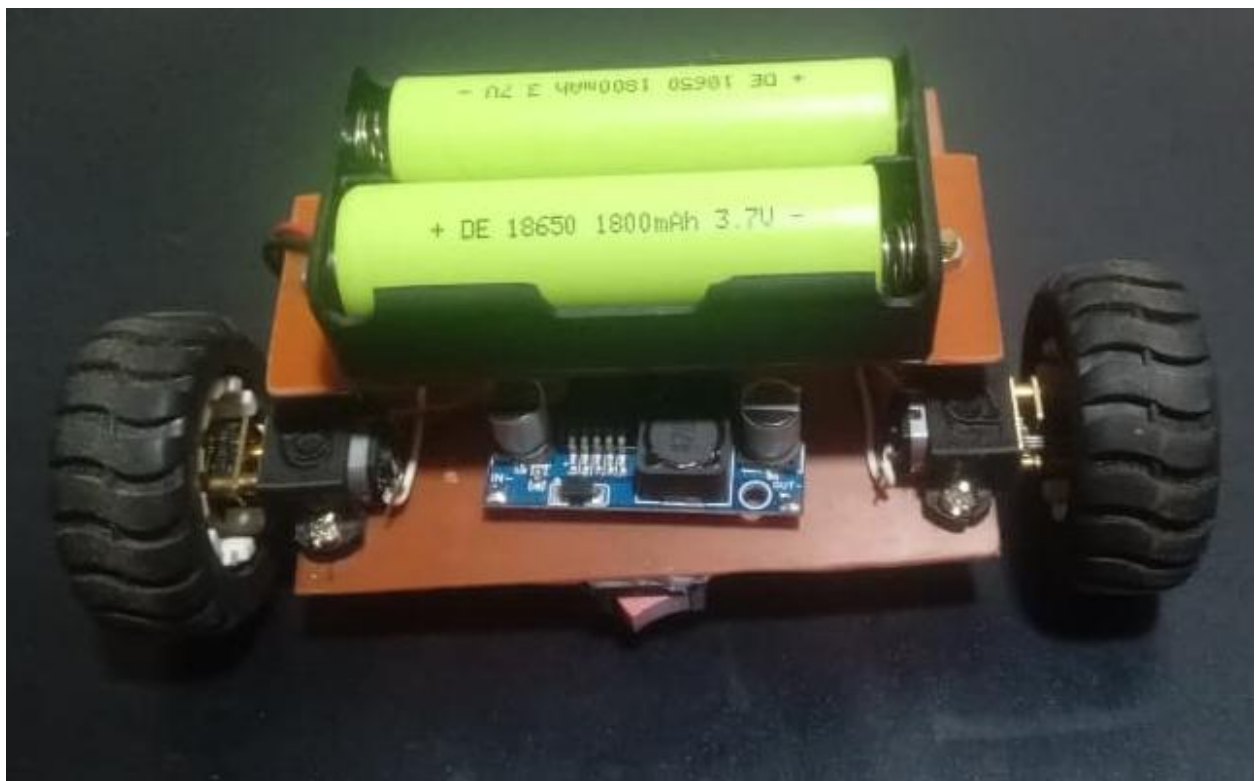
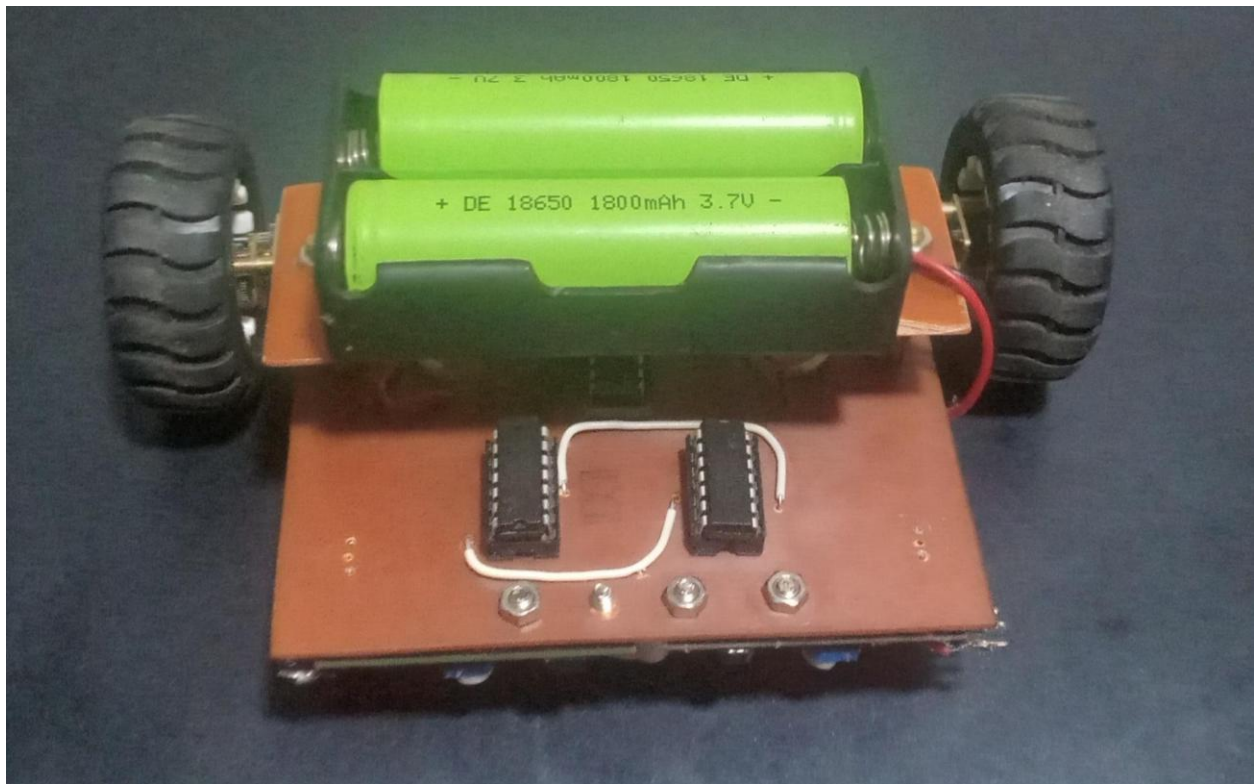


Figure 11.8: Final line follower robot